

Programlama Dilleri

Dr. Öğr. Üyesi Hayri Volkan Agun

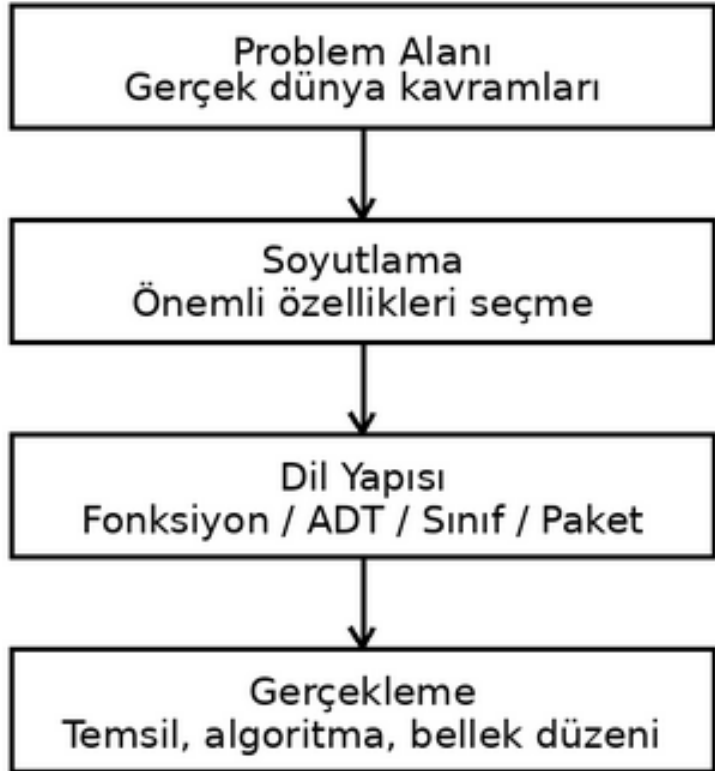
Bilgisayar Mühendisliği Bölümü

Bursa Teknik Üniversitesi

Veri Soyutlama ve Kapsülleme Yapıları

- Bu bölümde programlama dillerinin veri soyutlamayı nasıl desteklediği incelenir.
- Veri soyutlama, son 50 yılda programlama metodolojileri ve dil tasarımı açısından en önemli gelişmelerden biridir.
- Özellikle büyük yazılım sistemlerinde karmaşıklığı azaltır, temsil ayrıntılarını gizler ve program parçalarının daha güvenilir biçimde kullanılmasını sağlar.
- Soyutlama (abstraction), bir varlığın yalnızca en önemli özelliklerini dikkate alan temsil biçimidir. Örneğin “kuş” kavramı iki kanat, iki ayak, kuyruk ve tüy gibi ortak nitelikleri içerir. “Karga” dediğimizde bu ortak nitelikleri tekrar belirtmeyiz; yalnızca kargayı ayıran özelliklere odaklanırsınız.
- Programlama dillerinde soyutlama, programcının düşük seviyeli ayrıntıları geçici olarak göz ardı ederek daha üst düzey kavramlarla çalışmasını sağlar.
- Soyutlama, ayrıntıları tamamen yok etmek değildir; doğru ayrıntı düzeyini seçmektir.

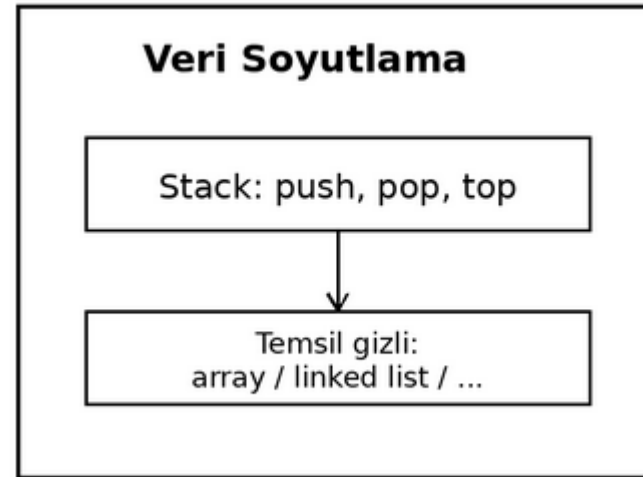
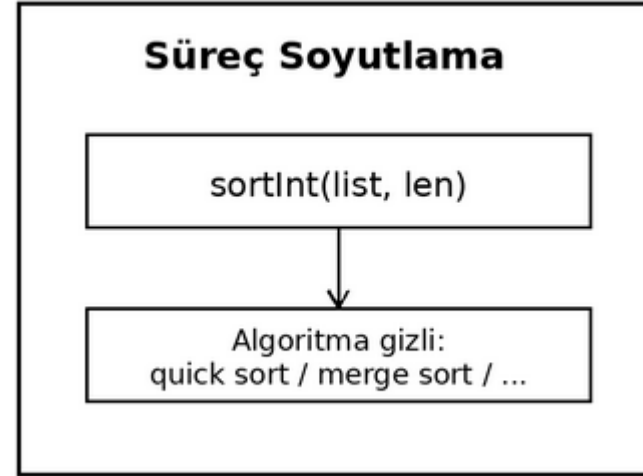
Veri Soyutlama ve Kapsülleme Yapıları



- Soyutlama (abstraction), bir varlığın yalnızca en önemli özelliklerini dikkate alan temsil biçimidir. Örneğin “kuş” kavramı iki kanat, iki ayak, kuyruk ve tüy gibi ortak nitelikleri içerir. “Karga” dediğimizde bu ortak nitelikleri tekrar belirtmeyiz; yalnızca kargayı ayıran özelliklere odaklanılır.
- Programlama dillerinde soyutlama, programcının düşük seviyeli ayrıntıları geçici olarak göz ardı ederek daha üst düzey kavramlarla çalışmasını sağlar.
- Soyutlama, ayrıntıları tamamen yok etmek değildir; doğru ayrıntı düzeyini seçmektir.

Soyutlama Türleri

- Çağdaş programlama dillerinde iki temel soyutlama vardır:
- **1. Süreç soyutlama (process abstraction):**
Bir işlemin nasıl yapıldığını gizler. Fonksiyonlar, prosedürler ve alt programlar bu kapsamdadır.
- **2. Veri soyutlama (data abstraction):**
Verinin nasıl temsil edildiğini gizler. Kullanıcı yalnızca veri üzerinde yapılabilecek işlemleri görür.
- Bu iki soyutlama türü birbirinden bağımsız değildir. Veri soyutlama, kendi işlemlerini tanımladığı için süreç soyutlamasına da dayanır.



Süreç Soyutlama

- `sortInt(list, listLen);`

Kullanıcının bilmesi gerekmeyen ayrıntılar:

Quick sort mu kullanılıyor?

Merge sort mu kullanılıyor?

Geçici bellek ayrılıyor mu?-

Karşılaştırma işlemi nasıl optimize ediliyor?

- Bu çağrı, sıralama işleminin soyutlanmış halidir.
- Kullanıcı yalnızca şunları bilir:
 - - list: sıralanacak dizi
 - listLen: dizinin uzunluğu
 - sortInt çağrısı sonunda dizinin sıralanacağı
- Bu örnekte algoritmanın kendisi saklanır. Kullanıcı yalnızca fonksiyonun adı ve protokolü üzerinden işlem yapar.

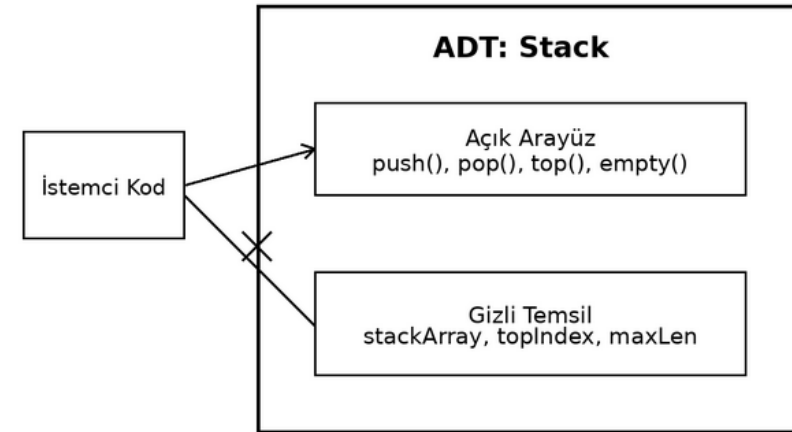
Veri Soyutlama

- Veri soyutlama, veri temsilini ve o temsil üzerinde çalışan işlemleri birlikte ele alan programlama dilidir.
- Bir soyut veri tipinde kullanıcı şunu görür:
 - Tip adı + işlemler
- Fakat şunu görmez:Verinin bellekteki gerçek temsili.
- Örneğin stack şu işlemleri sunar:
 - push
 - pop
 - top
 - empty
- Ama stack'in diziyle mi, bağlı listeye mi, yoksa dinamik bellekle mi gerçekleştirildiği istemciden saklanır.

ADT

- **Abstract Data Type (ADT)**, bir veri tipinin temsilini ve o veri üzerinde çalışan işlemleri tek bir kapsül içinde tanımlar.
- Bir ADT şu koşulları sağlamalıdır:
 - Nesnelerin temsili istemcilerden gizlenir.
 - Nesneler yalnızca tanımlanan işlemlerle değiştirilebilir.
 - Tip bildirimi ve işlem protokolleri tek bir sözdizimsel birimde yer alır.
 - Arayüz, gerçek temsil ve algorithmadan bağımsızdır.

- ADT, veri yapısını yalnızca bellek düzeni olarak değil, davranışsal bir tip olarak tanımlar.



Yerleşik Tipler

- Yerleşik veri tipleri de soyut veri tipi gibi düşünülebilir.
- Örneğin float tipi:
 - Değer saklar.
 - Aritmetik işlemler sağlar.
 - Bellekte belirli bir temsil biçimi vardır.
 - Kullanıcı bu temsili doğrudan bilmez.
- Kullanıcı float değerinin bit düzeyindeki gösterimiyle uğraşmaz.
- Bu durum taşınabilirliği artırır.
- Farklı donanım temsilleri olsa bile programcı aynı dil soyutlamasıyla çalışır.

`// C`

```
float x = 3.14;  
float y = x * 2.0;
```

Bilgi Saklama

- **Information hiding**, veri temsilinin istemci programlardan saklanmasıdır.
- Örneğin stack içinde şu alanlar bulunabilir:
 - stackArray
 - topIndex
 - maxLen

- Fakat istemci bunlara doğrudan erişmemelidir.
- Yanlış tasarım:

```
</> Java
```

```
myStack.topIndex = -100;
```

- Doğru tasarım:

```
</> Java
```

```
myStack.push(42);
```

```
myStack.pop();
```

Bilgi Saklama

- Bilgi gizleme şu açılardan önemlidir:
 - Güvenilirliği artırır.
 - Hatalı doğrudan erişimi engeller.
 - Kodun okunabilirliğini artırır.
 - İsim çakışmalarını azaltır.
 - Temsil değişikliklerini istemci koddan izole eder.
- Bu, ADT'nin en önemli yazılım mühendisliği katkısıdır: temsil değişebilir, fakat protokol sabit kalır.

- Örnek temsil değişikliği:
 - Eski temsil: linked list
 - Yeni temsil: array
- Eğer arayüz aynı kalırsa istemci kod değişmez:

```
push(stk, x);  
pop(stk);  
top(stk);
```

Getter ve Setter

- Bazı durumlarda istemcinin gizli verilere kontrollü erişmesi gerekir.
- Bu durumda accessor methods kullanılır.
- `getAge` okuma erişimi sağlar. `setAge` ise değer atama sırasında kısıt koyar.
- Fakat her veri alanı için otomatik getter/setter yazmak ADT ilkesini zayıflatabilir. Çünkü nesnenin iç durumu dolaylı olarak dışarı açılmış olur.

`//` Java

```
public int getAge() {  
    return age;  
}  
  
public void setAge(int value) {  
    if (value >= 0)  
        age = value;  
}
```

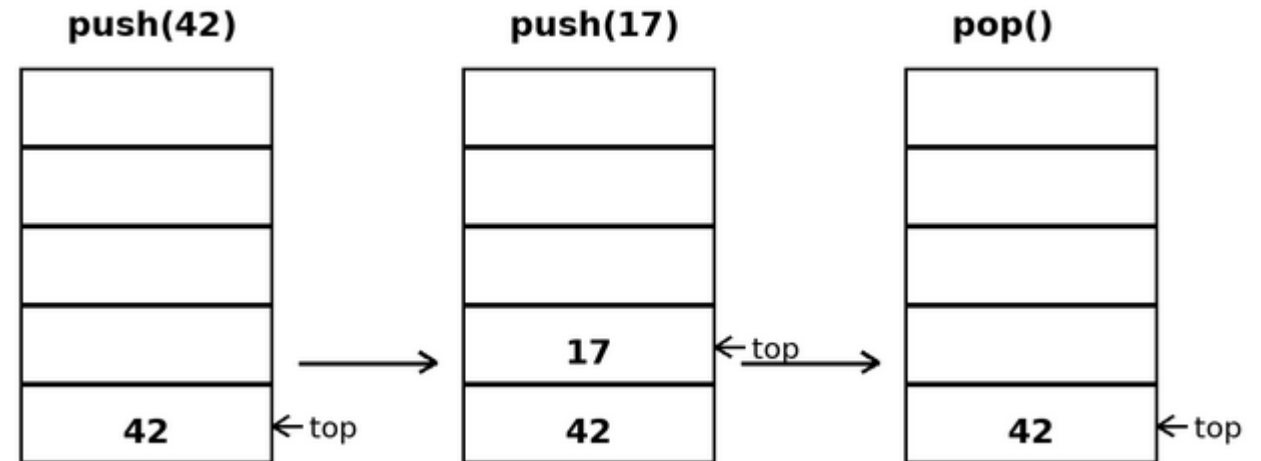
Zayıf tasarım:

`//` Java

```
account.setBalance(50);
```

ADT Örnek

- Stack, yalnızca tepe elemanına erişime izin veren doğrusal veri yapısıdır.
 - create
 - push
 - pop
 - top
 - empty
 - destroy
- Bazı dillerde create ve destroy açıkça yazılmaz; nesne bildirimi ve kapsam sonu bu işlemleri otomatik hale getirir.
 - create(stk1);
 - push(stk1, color1);
 - push(stk1, color2);
 - temp = top(stk1);
- Bu kod stack'in nasıl saklandığını bilmez. Sadece ADT arayüzünü kullanır.

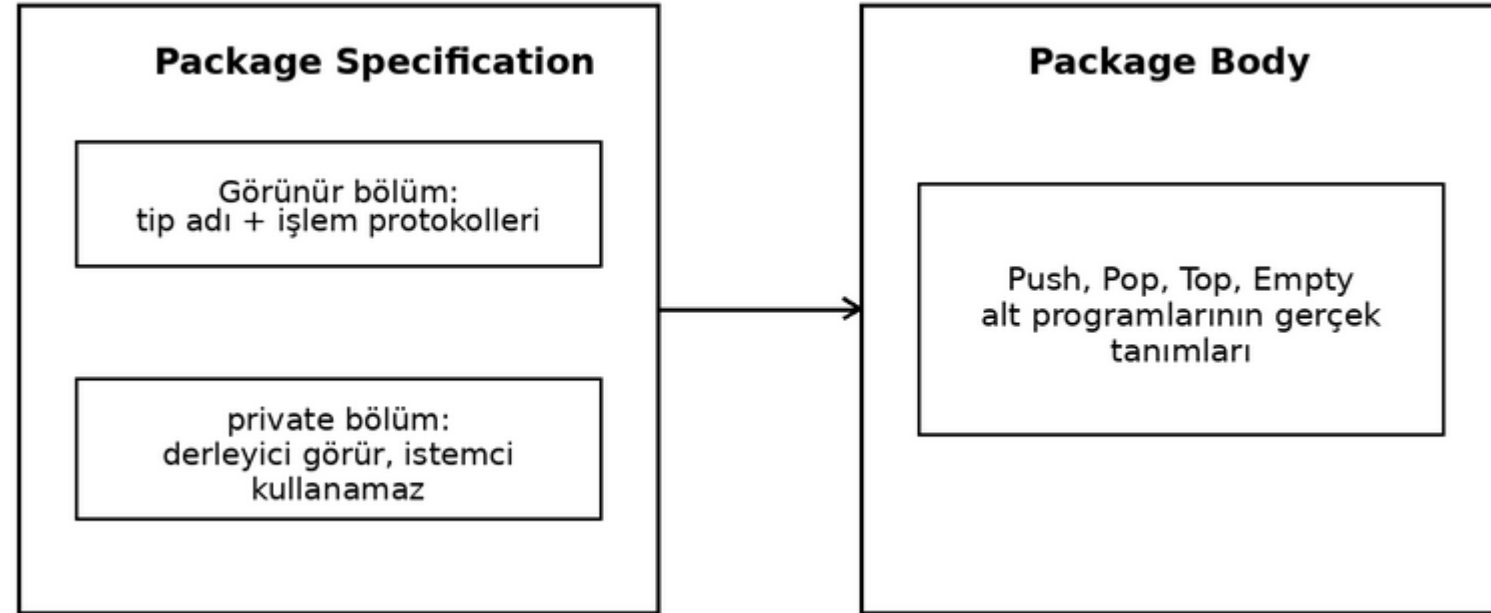


ADT

- Bir dil ADT desteği sunarken şu sorulara cevap vermelidir:
 - Tip arayüzü hangi kapsül içinde tanımlanır?
 - Temsil nasıl gizlenir?
 - Hangi erişim denetimleri vardır?
 - Tip parametreleştirilebilir mi?
 - Arayüz ve gerçekleştirme fiziksel olarak ayrılabilir mi?
- Bu sorular dil tasarımını doğrudan etkiler. Ada package, C++ class, Java class, C# class/struct, Ruby ise dinamik class yapısı üzerinden farklı çözümler üretir.

Ada

- Ada'da ADT desteği **package** yapısıyla sağlanır.
- Bir package iki parçadan oluşabilir:
 - package specification
 - package body
- Specification, istemciye görünen arayüzü tanımlar. Body, işlemlerin gerçek tanımlarını içerir.



Ada

```
package Stack_Pack is
  type Stack_Type is limited private;
  Max_Size : constant := 100;

  function Empty(Stk : in Stack_Type) return Boolean;
  procedure Push(Stk : in out Stack_Type;
                 Element : in Integer);
  procedure Pop(Stk : in out Stack_Type);
  function Top(Stk : in Stack_Type) return Integer;

private
  type List_Type is array (1..Max_Size) of Integer;
  type Stack_Type is record
    List : List_Type;
    Topsub : Integer range 0..Max_Size := 0;
  end record;
end Stack_Pack;
```

⟨⟩ ada

```
package body Stack_Pack is

  function Empty(Stk: in Stack_Type) return Boolean is
  begin
    return Stk.Topsup = 0;
  end Empty;

  procedure Push(Stk : in out Stack_Type;
                 Element : in Integer) is
  begin
    if Stk.Topsup >= Max_Size then
      Put_Line("ERROR - Stack overflow");
    else
      Stk.Topsup := Stk.Topsup + 1;
      Stk.List(Stk.Topsup) := Element;
    end if;
  end Push;

end Stack_Pack;
```

C++

```
// C++  
  
class Stack {  
private:  
    int *stackPtr;  
    int maxLen;  
    int topSub;  
  
public:  
    Stack();  
    ~Stack();  
    void push(int);  
    void pop();  
    int top();  
    int empty();  
};
```

- private bölüm temsil ayrıntısını içerir. public bölüm ise istemcinin kullanabileceği arayüzdür.
- C++'ta sınıf aynı zamanda bir tiptir. Bu yönüyle Ada package yapısından farklıdır; Ada package daha genel bir kapsülleme mekanizmasıdır.

C++ Constructor ve Destructor

```
// C++  
  
Stack::Stack() {  
    stackPtr = new int[100];  
    maxLen = 99;  
    topSub = -1;  
}  
  
Stack::~~Stack() {  
    delete [] stackPtr;  
}
```

- Constructor, nesne oluşturulduğunda çağrılır ve ilk durumu hazırlar. Destructor, nesnenin yaşam süresi bittiğinde heap belleği geri verir.
- Bu C++ için kritiktir çünkü stack nesnesi kendisi otomatik kapsamda olabilir, fakat içinde heap belleğe işaret eden pointer bulunabilir.

C++ İstemci Kodu

❏ C++

```
int main() {  
    int topOne;  
  
    Stack stk;  
    stk.push(42);  
    stk.push(17);  
  
    topOne = stk.top();  
    stk.pop();  
  
    return 0;  
}
```

- İstemci stk.push, stk.top ve stk.pop gibi public metotları kullanır. Aşağıdaki alanlara erişemez:
 - stk.stackPtr;
 - stk.maxLen;
 - stk.topSub;
- Böylece veri temsili sınıf içinde saklanır. ADT arayüzü nesne üzerinden çağrılır.

C++ Arayüz

```
// C++

class Stack {
private:
    int *stackPtr;
    int maxLen;
    int topSub;

public:
    Stack();
    ~Stack();
    void push(int);
    void pop();
    int top();
    int empty();
};
```

```
// C++

#include "Stack.h"

void Stack::push(int number) {
    if (topSub == maxLen)
        cerr << "stack is full";
    else
        stackPtr[++topSub] = number;
}
```

- Header dosyası arayüzü, .cpp dosyası gerçeklemeyi taşır. Bu ayırım ayrı derleme ve modülerlik açısından önemlidir.

Java

- Java'da ADT desteği sınıflar üzerinden sağlanır. Tüm nesneler heap üzerinde oluşturulur ve referanslarla erişilir.
- Java'da constructor nesneyi başlatır. C++'tan farklı olarak destructor yoktur; bellek yönetimi garbage collection tarafından yapılır.

Java

```
class StackClass {  
    private int[] stackRef;  
    private int maxLen;  
    private int topIndex;  
  
    public StackClass() {  
        stackRef = new int[100];  
        maxLen = 99;  
        topIndex = -1;  
    }  
}
```

Java

```
</> Java

public void push(int number) {
    if (topIndex == maxLen)
        System.out.println("Error in push");
    else
        stackRef[++topIndex] = number;
}

public void pop() {
    if (empty())
        System.out.println("Error in pop");
    else
        --topIndex;
}

public boolean empty() {
    return topIndex == -1;
}
```

- push yeni elemanı tepeye ekler. pop tepe elemanını çıkarır. empty stack'in boş olup olmadığını döndürür.
- Temsil alanları private olduğu için istemci bunları doğrudan değiştiremez.

Java İstemci

```
</> Java

public class TstStack {
    public static void main(String[] args) {
        StackClass myStack = new StackClass();

        myStack.push(42);
        myStack.push(29);

        System.out.println(myStack.top());

        myStack.pop();
        System.out.println(myStack.top());
    }
}
```

- İstemci yalnızca public metotları kullanır.
- Java'da sınıf tanımı içinde metot gövdelerinin bulunması gerekir.
- Bu nedenle Java'da ADT bildirimi ve gerçekleştirme çoğu zaman aynı sözdizimsel birimde yer alır.

C Sharp

```
</> C#  
  
public class Weather {  
    private int degreeDays;  
  
    public int DegreeDays {  
        get {  
            return degreeDays;  
        }  
        set {  
            if (value < 0 || value > 30)  
                Console.WriteLine("Value is out of range");  
            else  
                degreeDays = value;  
        }  
    }  
}
```

- C# da Java gibi sınıfları heap üzerinde oluşturur. Ayrıca property mekanizması güçlüdür.
- DegreeDays dışarıdan public alan gibi görünür. Fakat erişim aslında getter ve setter üzerinden yapılır.
- Setter içinde kısıt kontrolü yapılır.

Ruby

```
<> Ruby

class StackClass
  def initialize
    @stackRef = Array.new(100)
    @maxLen = 100
    @topIndex = -1
  end

  def push(number)
    @topIndex = @topIndex + 1
    @stackRef[@topIndex] = number
  end
end
```

- Ruby’de ADT desteği sınıflarla sağlanır.
- Ruby’de örnek değişkenleri @ ile başlar.
- Tüm veri üyeleri private kabul edilir.
- Bu, veri gizleme açısından güçlü bir özelliktir.

Ruby

```
#!/usr/bin/ruby

myStack = StackClass.new

myStack.push(42)
myStack.push(29)

puts "Top element is: #{myStack.top}"

myStack.pop
```

- Ruby dinamik tipli olduğu için aynı stack farklı tiplerden nesneler saklayabilir.

```
#!/usr/bin/ruby

myStack.push(42)
myStack.push("hello")
myStack.push(3.14)
```

- Bu esneklik statik tipli dillere göre daha fazladır. Fakat tür hataları derleme zamanında değil, çalışma zamanında ortaya çıkabilir.

Ruby

- Ruby’de bir sınıf sonradan genişletilebilir.
- Aynı sınıf adıyla ikinci bir tanım yapılabilir ve sınıfa yeni metot eklenir. Bu özellik esneklik sağlar.
- Ancak akademik açıdan önemli dezavantaj şudur:
 - Okunabilirlik ve güvenilirlik azalabilir.
- Çünkü bir sınıfın gerçek davranışını anlamak için program boyunca yapılan tüm ek tanımlar incelenmelidir.

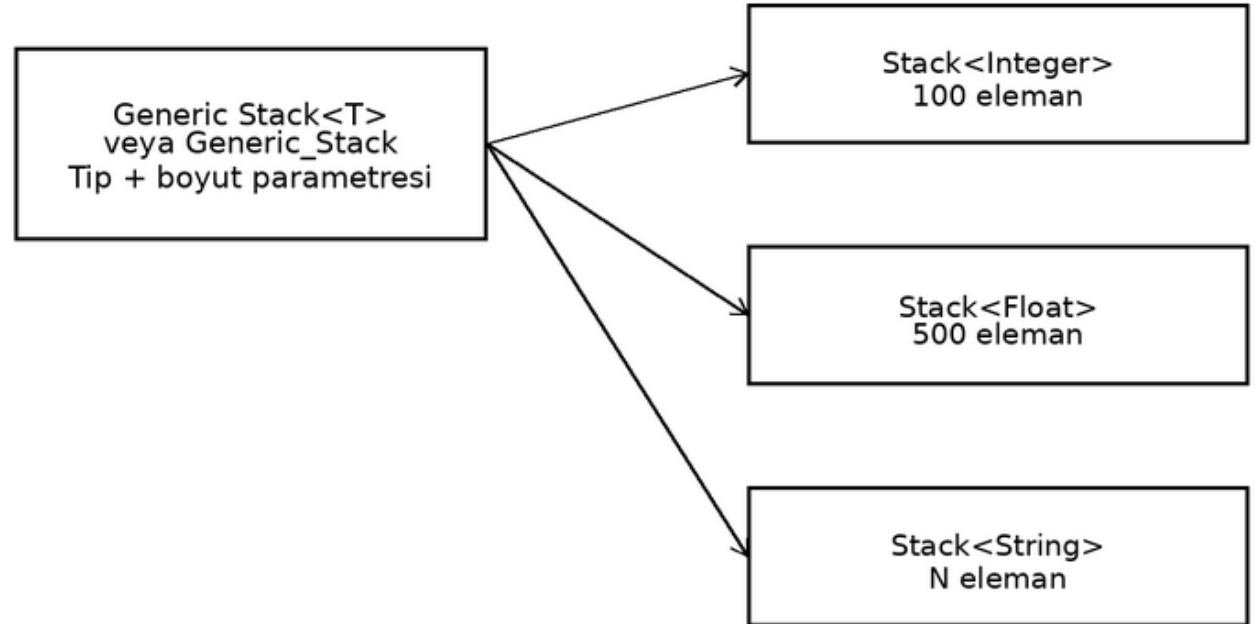
❏ Ruby

```
class MyClass
  def meth1
    puts "first method"
  end
end
```

```
class MyClass
  def meth2
    puts "second method"
  end
end
```

Parametrelili ADT

- Statik tipli dillerde aynı ADT'yi farklı veri tipleri için yeniden yazmak istemeyiz.
 - IntegerStack
 - FloatStack
 - StringStack
- Parametrelili ADT, aynı tasarımın farklı eleman tipleriyle kullanılmasını sağlar.
 - Stack<Integer>
 - Stack<Float>
 - Stack<String>
- Bu yaklaşım kod tekrarını azaltır, tip güvenliğini artırır ve yeniden kullanılabilirliği yükseltir.



C++

Type template parametresidir. Derleme zamanında int, float, string gibi somut tiplere dönüştürülür.

```
⟨/⟩ C++
```

```
Stack<int> intStack(100);  
Stack<double> doubleStack(50);
```

```
⟨/⟩ C++
```

```
template <typename Type>  
class Stack {  
private:  
    Type *stackPtr;  
    int maxLen;  
    int topSub;  
  
public:  
    Stack(int size) {  
        stackPtr = new Type[size];  
        maxLen = size - 1;  
        topSub = -1;  
    }  
  
    void push(Type value) {  
        if (topSub == maxLen)  
            cout << "stack is full";  
        else  
            stackPtr[++topSub] = value;  
    }  
  
    Type top() {  
        return stackPtr[topSub];  
    }  
};
```

Java

```
</> Java
```

```
Stack2<String> myStack = new Stack2<String>();  
myStack.push("Ada");  
myStack.push("Java");
```

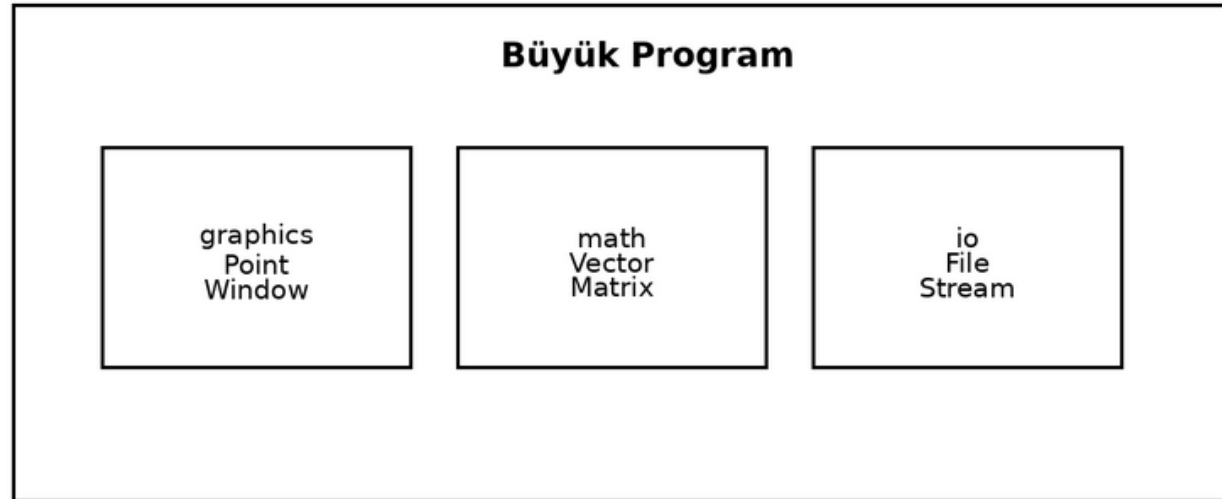
Java 5.0 ile generic sınıflar gelmiştir. Bu sayede cast ihtiyacı azalır ve yanlış tipte eleman ekleme derleme zamanında yakalanır.

```
</> Java
```

```
import java.util.*;
```

```
public class Stack2<T> {  
    private ArrayList<T> stackRef;  
    private int maxLen;  
  
    public Stack2() {  
        stackRef = new ArrayList<T>();  
        maxLen = 99;  
    }  
  
    public void push(T newValue) {  
        if (stackRef.size() == maxLen)  
            System.out.println("stack is full");  
        else  
            stackRef.add(newValue);  
    }  
  
    public T top() {  
        return stackRef.get(stackRef.size() - 1);  
    }  
}
```

İsimlendirme ve Kapsülleme



`// C++`

```
namespace math {  
    class Vector {};  
}
```

```
namespace graphics {  
    class Vector {};  
}
```

`// C++`

```
math::Vector v1;  
graphics::Vector v2;
```